

정종윤

Phone: 010-3558-4676

Email: jongyun.jeong.00@gmail.com

GitHub: <https://github.com/BellYun>



SUMMARY

- React 및 Next.js 기반 풀스택역량을 바탕으로, 서버 API 설계, 모바일 앱 환경까지 아우르는 End-to-End 서비스 구축 경험을 보유하고 있습니다.
- 성능 최적화와 사용자 경험을 최우선하는 프론트엔드 개발자로서, 렌더링 병목 지점을 데이터 기반으로 분석하고 개선한 경험이 있습니다
- AI를 단순 활용하는 것에 그치지 않고, 사내 정책 자동화 및 코드베이스 분석 봇을 구축하는 등 **기술을 통한 운영 오버헤드 감소**에 주력합니다
- **스타트업 인턴과 DAU 700+** 규모의 서비스를 1인 운영한 경험으로, 기술적 의사결정이 실제 사용자 지표에 미치는 영향을 고민해왔습니다.

EXPERIENCE

WAKAWAKA / 인턴 [[App](#)]

2025.12 - 2026.03

채팅을 통한 크리에이터 협업 매칭 서비스 / FE 2인, BE 4인

React Native, TypeScript, Tanstack Query, Tailwind CSS

- **A/B 테스트 설계, 운영을 통한 데이터 기반 UX 의사결정.**
 - **Amplitude** 퍼널 분석에서, 첫 메시지 전송율이 낮은 원인을 추적한 결과. 유저가 상대의 정보를 깊게 판단하지 않고 스와이프하는 문제를 발견. 카드에 정보를 추가하면 판단 품질이 올라간다는 가설을 수립, **A/B 테스트**를 설계해 14일간 운영.
 - 그 결과, 프로필 조회 후 좋아요 전환율 18% → 27%, 첫 메시지 전송율 32.4% → 41.2%의 개선 추이를 확인하고 RN 버전에 반영
- **AsyncStorage 기반 네트워크 실패 복원 큐를 통한 인터랙션 유실 방지.**
 - 기존 낙관적 업데이트 구조에서 네트워크 실패 시 좋아요/싫어요가 유실되는 문제를 발견하였습니다. 실패한 요청을 AsyncStorage 기반 재시도 큐에 적재하고, NetInfo 이벤트로 네트워크 복구를 감지하여 FIFO 순서로 자동 재전송하는 레이어를 구현했습니다.
 - 최대 5회 재시도(5초 간격)로 재시도 횟수를 제한하여 무한 루프를 방지하고, 최종 실패 시 사용자에게 수동 재시도 UI를 노출했습니다.
- **Turborepo 기반 모노레포 도입으로 Web/App 코드 중복 제거.**
 - Web과 App이 각자 API 타입과 비즈니스 로직을 따로 관리하는 구조에서 서비스 확장 시 명세 불일치를 식별하고, 모노레포를 구축했습니다.
 - UI 컴포넌트는 렌더링 환경 차이로 제외하고, API 타입·유틸만 shared 패키지로 통합해 명세 변경 대응 포인트를 단일화 했습니다.
- **Slack Bolt + Claude API 기반 사내 업무 보조 봇 개발.**
 - 비개발 직군이 유저 문의를 1차 판단 할 수 있도록 코드베이스, 정책 기반 컨텍스트를 자동 제공하는 봇을 개발하였습니다.
 - 이로 인해 개발자에 대한 반복적인 확인 요청을 줄이고, 문의 전달시 구체성을 높여 커뮤니케이션 오버헤드를 감소했습니다.

PROJECTS

ER&GG [[Web](#)] [[Github](#)]

2026.02 - 현재

1인 개발로 진행한 게임 통계 분석 사이트, 일평균 700명 이상의 사용자(DAU) 유지

Next.js, TypeScript, Zustand, TailwindCSS

- **고비용 통계 집계 연산 최적화 및 DB 부하 격리.**
 - 다중 테이블 조인이 포함된 시너지 조합 API의 높은 계산 비용으로 인해 지연시간 발생, Cache-Aside 패턴 기반 Redis 캐싱 전략을 수립하여 응답 시간을 **400ms에서 5.4ms로 단축**하였습니다.
- **Server/Client Component 경계 설계 기반 Streaming SSR 최적화.**[블로그]
 - 데이터 의존성이 있는 컴포넌트는 Server Component로, 상호작용이 필요한 UI만 Client Component로 분리하여 번들 크기를 절감했습니다.
 - Suspense 경계별 독립 스트리밍으로 병렬 데이터 로딩을 구현하고, React.cache()로 동일 렌더 사이클 내 중복 DB 쿼리를 제거했습니다..
- **generateStaticParams 기반 SSG + ISR 동적 라우트 아키텍처 설계.** [블로그]
 - 87개 캐릭터 상세 페이지를 동적 라우트로 전환하고, generateStaticParams 사전 빌드를 통해 TTI를 (12.9s->6.3s)단축했습니다.
 - 패치 주기 기반의 유저 의사결정 흐름을 고려해 ISR(1시간)을 채택, 서버 비용 절감과 데이터 신선도를 동시에 확보했습니다.
- **Edge Runtime Proxy 기반 API 트래픽 제어 및 보안 강화.**
 - 경로별 차등 Rate Limiting(인증 10회/분, 일반 60회/분)을 Edge Runtime에서 처리하여 Origin 서버 도달 전 불필요한 요청을 차단
 - XSS·Clickjacking·MIME Sniffing 방어를 위한 Security Headers를 전 경로에 일괄 적용하여 보안 표면을 축소했습니다

Rexume [[Github](#)]

2025.10 - 2025.12

개발자들의 이력서 공유 및 피드백 플랫폼 / FE: 3인 BE: 4인

React, TypeScript, Zustand, TailwindCSS, Vitest

● 저사양 기기 타겟 렌더링 최적화. [\[블로그\]](#)

- Lighthouse CPU 6x 환경(저사양 모바일 환경)에서 메인스레드에서 발생하는 PDF 렌더링으로 인한 350ms의 TBT 발생.
- OffscreenCanvas + Web Worker 기반 렌더링 파이프라인으로 Canvas 2D 드로잉을 메인 스레드에서 완전히 분리했습니다.
- Puppeteer를 활용한 성능 측정 결과 TBT 63%(350ms → 129ms) 개선 및 저사양 기기에서의 입력 반응성 확보.

● Sentry 및 계층형 Error Boundary 도입을 통한 서비스 신뢰성 강화.

- Sentry와 3단계 Error Boundary를 활용한 에러 격리 및 복구 체계 구축.
- Sentry 기반 실시간 모니터링과 Graceful Degradation 전략을 적용하여 장애 상황에서도 안정적인 UX 유지.

● 테스트 코드 기반의 협업 프로세스 및 CI 파이프라인 구축. [\[블로그\]](#)

- 팀원 간 요구사항 및 UI 플로우에 대한 해석 차이로 인해 PR 단계에서 반복적인 수정 작업이 발생하고 개발 효율이 저하되었습니다.
- 구현 전 단위 테스트를 기능 명세서로 활용해 동작 기준을 코드로 고정하고, CI 자동화를 통해 합의된 동작만 병합되도록 재발 방지 시스템을 구축.

HangOut [[Github](#)]

2024.07 - 2024.12

1인 여행자를 위한 동행 찾기 서비스 / FE: 2인 BE: 2인

React, Tanstack-Query, TypeScript, MSW, Styled-components

● 브라우저 Critical Rendering Path 최적화를 통한 LCP 지표 37.5% 개선 [\[블로그\]](#)

- Chrome DevTools Performance Trace로 컴포넌트 마운트 시점에 종속된 히어로 이미지의 로딩 병목 식별
- 히어로 이미지에 Fetch Priority 적용 및 컴포넌트 마운트 의존성 제거를 통한 리소스 수집 우선순위 최적화, 페이지 초기 진입시 불필요한 라이브러리에 Dynamic Import 적용하여 분리, 핵심 비주얼 로드 후 라이브러리를 준비하도록 구현
- Lighthouse Desktop LCP를 37.5%(3.2s → 2.0s) 단축하여, 초기 진입 경험을 개선했습니다.

● 데이터 페칭 전략을 통한 인터랙티브 UX 구현.

- 인기글 목록에 staleTime 적용, 재방문 시 캐시 데이터를 즉시 렌더링하고.
- 좋아요·북마크 기능에 TanStack Query Optimistic Update를 적용해 API 응답 전 UI를 선제 반영했습니다. 네트워크 지연 시에도 인터랙션 피드백이 즉시 표시되어 끊김 없는 UX를 제공하고, onError/onSettled 기반 롤백으로 실패 시 상태를 자동 복구했습니다.

SKILLS

- Languages & State: TypeScript 5, JavaScript, TanStack Query 4, Zustand
- Test / Tooling: Vitest, React Testing Library, Jest | ESLint, Prettier
- Server : NestJS
- DevOps: GitHub Actions, Vercel, Docker, AWS

ACTIVITY

오픈소스 기여모임 10기

2026.01 ~ 2026.02

- 목표 : 대규모 PDF 메타데이터 로딩 시 성능 향상을 위한 Batching API 제안
- 과정 : 기존 Streaming 방식의 한계를 가설로 설정, 데이터 일괄 처리를 통한 최적화 시도
- 내용 : 메인테이너 리뷰를 통해 프로젝트의 설계를 이해하였으며, 벤치마킹 결과 저사양 기기에서 직렬화 비용으로 인해 성능이 1.8배 저하됨을 확인
- 성과 : 대규모 시스템에서 기술 도입 시 단순 수치보다 아키텍처 적합성과 환경 특화 의사결정이 중요함을 체득 [\[블로그\]](#)

EDUCATION

한국공학대학교 / 컴퓨터공학부

2019.03 ~ 2025.08